Concurrent Programming with Go

Concepts of Programming Languages

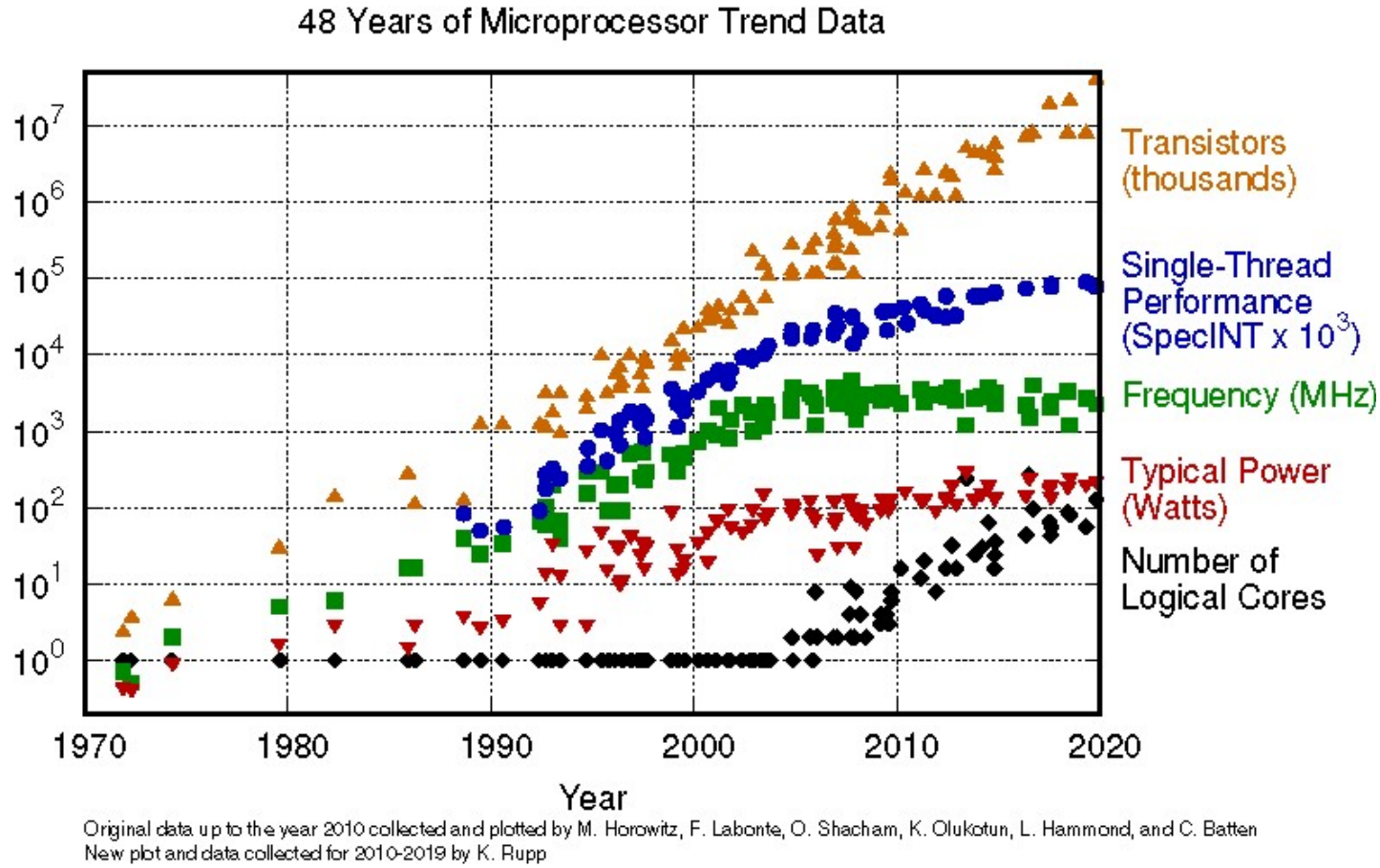
Sebastian Macke
Rosenheim Technical University

Last lecture

- Pure Functional programming
- Introduction to Haskell
- Types, partial application, control flow, lists, filters, ...
- Implicit concurrency

Why Concurrent Programming?

- Computer clock rates do not get higher anymore (since 2004!)
- But Moores Law is still valid (Multicore!)



The modern world is parallel

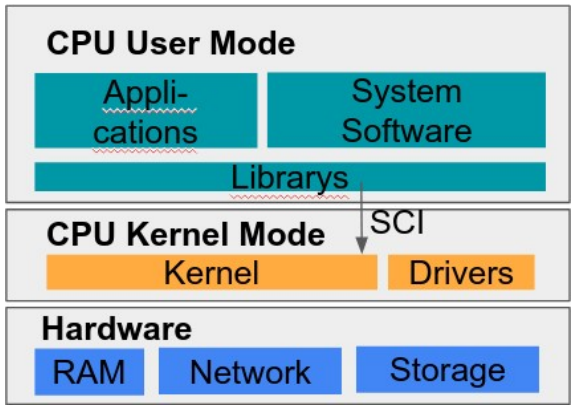
- Multicore
- Networks
- Clouds of CPUs
- Loads of users

Multiprocessor computer system

Shared memory systems are the most common types of multiprocessor computer system

- Two or more CPUs within one computer system.
- The processors of the multiprocessor system share a common memory.
- Communicate and sync via memory

What are Software or Kernel Threads

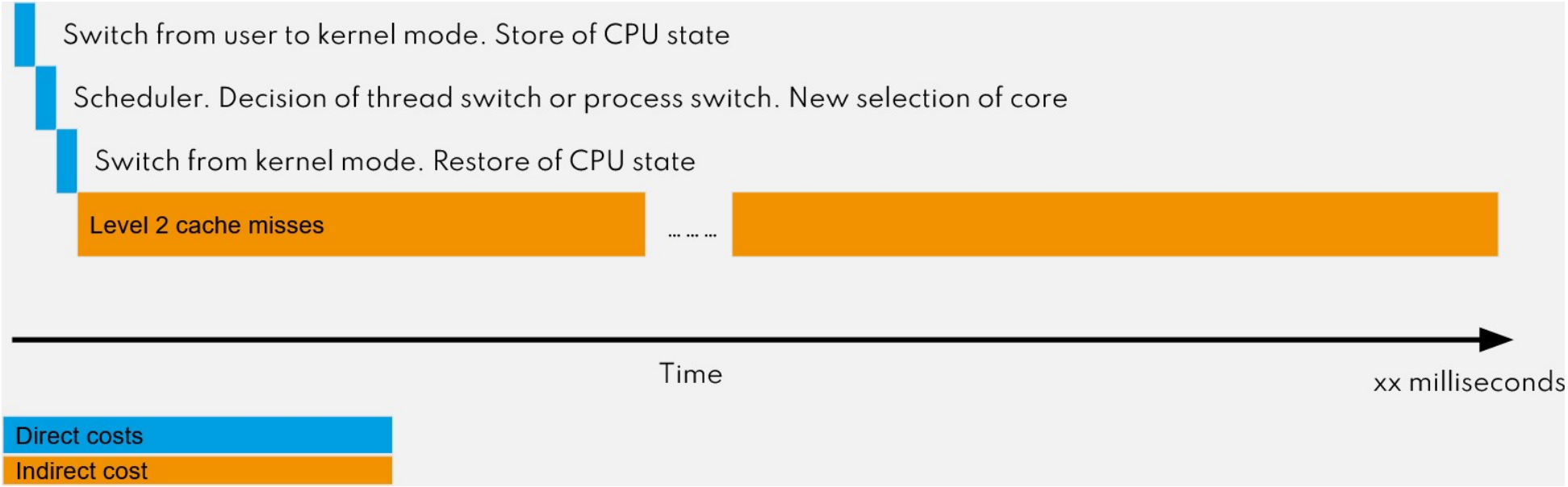


Kernel Mode: Highest Privilege level
Access to hardware via drivers, Control over RAM, File system, Memory
Kernel controls CPUs via a scheduler and provides threads to user space

User Mode: Lowest Privilege level
No direct hardware access, Calls the kernel via the System Call Interface (SCI)
All IO has to run through the kernel. E. g. A process cannot "wait" itself (Endless loop excluded).
A process can ask the kernel for threads

How expensive is a context switch

Example: Process switch for one CPU core



A task switch can take many µs up to 1ms. You should prevent context switches as much as possible.

7

Virtual Threads

8

Blocking calls to the kernel

- IO system calls from a thread can block for a very long time E. g. Access to the filesystem or waiting for the network data
- This is usually solved by thread pools. Hundreds of waiting software threads ready to do some work (e. g. Java)

```
ExecutorService executorService = Executors.newFixedThreadPool(10);
Future<String> future = executorService.submit(() -> "Hello World");
// some operations
String result = future.get();
```

- Threadpools help preventing the additional cost of creating a thread when needed, but does not solve the cost of context switches
- However the major kernels (Windows, Linux, MacOs, ...) allow for event queue driven system calls.

One interesting technique in this context are Coroutines.

Coroutines I

- Coroutines allow to jump in and out of functions at any time

JavaScript example

```
function* test() {
  var x = yield 'Hello!';
  var y = yield 'First I got: ' + x;
  return 'Then I got: ' + y
}

var tester = test()
console.log(tester.next())   // Output: Hello
// .. do something else
console.log(tester.next("cat")); // Output: First I got: cat
// .. do something else
console.log(tester.next("dog")); // Output: then I got: dog
```

When "yield" is executed the JavaScript engine has to store the state of the function.
Usually the program counter, call stack and CPU registers.

Coroutines II

JavaScript is single threaded. However doesn't allow to block on blocking calls.

- Old technique with callbacks

```
fetch('http://example.com/movies.json')
.then(response => response.status())  // is executed as callback
```

- New technique with async and await

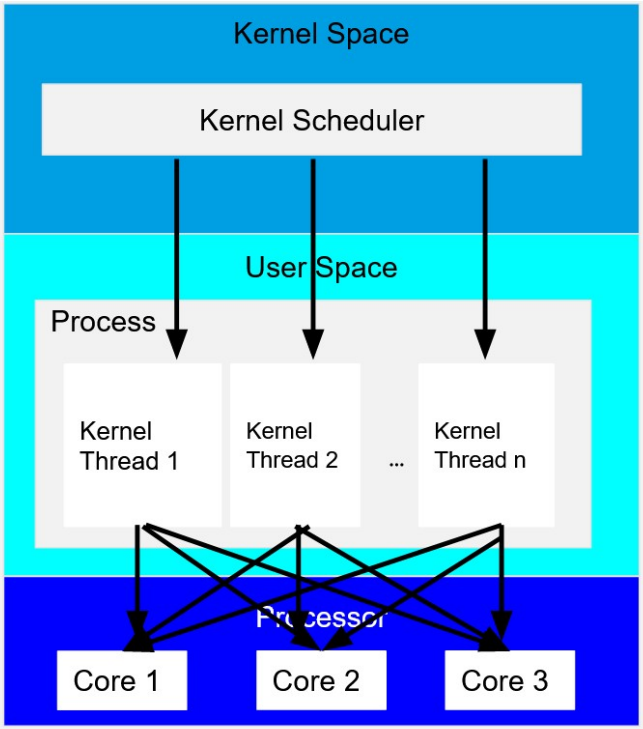
```
async function Download() {
    var response = await fetch('https://playcode.io/')
    console.log(response.status)
}

await Download()
```

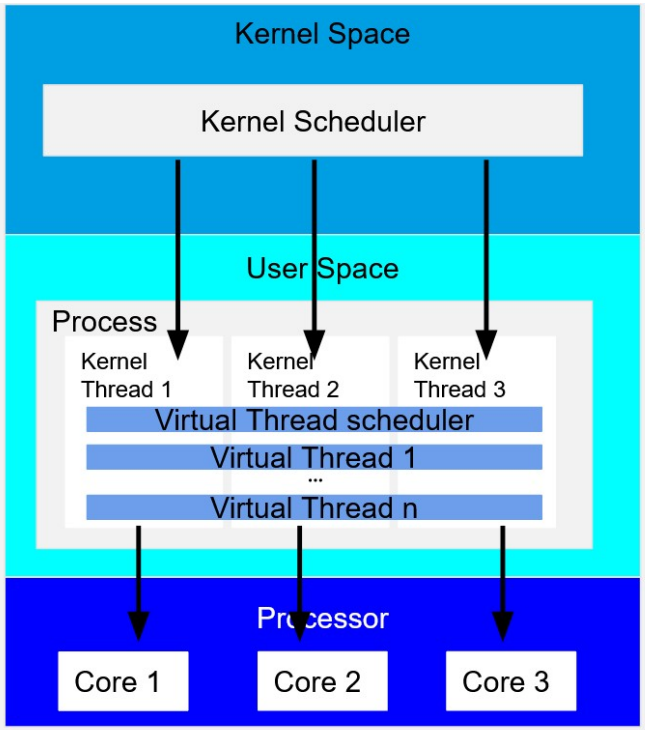
The technique of Coroutines is implicitly used for blocking calls.
Advantage: 1. No callback hell. 2. You just don't have to worry about blocking. Just write your code down.
But: This must be a language feature directly supported by their runtime

Go uses its own Thread Operating Model

- Kernel based scheduling (left), User Mode based scheduling (right)



Kernel scheduler organizes n threads



Kernel scheduler manages as many threads as there are cores. Each kernel thread is bound to one core. Witin these kernel threads, virtual threads are organized.

12

Virtual Threads

Goroutines are neither hardware nor software threads

- Goroutines are completely organized in user space.
- They behave like threads, but they're much cheaper. Instead of several hundred software threads you can have **even millions** of virtual threads.
- Go implements its own scheduler in user space
- Goroutines are multiplexed onto OS threads as required.
- When a goroutine blocks the thread will execute other goroutines via the same technique as used by coroutines.
- IO Calls and calls to the Go Standard Library trigger the scheduler

In case of independent threads and blocking calls you can just write your code down like it is single threaded!

Goroutines

A goroutine is a function running independently in the same address space as other goroutines

```
f("hello", "world") // f runs; we wait
```

```
go f("hello", "world") // f starts running
g() // does not wait for f to return
```

Like launching a function with shell's & notation.

One Million Parallel Threads

```
func Sleep() {
    time.Sleep(time.Duration(rand.Intn(1000)) * time.Millisecond)
}

func main() {
    fmt.Println("Number of Goroutines running: ", runtime.NumGoroutine())
    for i := 0; i < 1000000; i++ {
        go Sleep()
    }
    for i := 0; i < 15; i++ {
        fmt.Println("Number of Goroutines running: ", runtime.NumGoroutine())
        time.Sleep(100 * time.Millisecond)
    }

}
```

- Stacks start small, but grow and shrink as required.
- Goroutines aren't free, but they're very cheap.

Go provides:

- Concurrent execution (goroutines)
- Low level blocking primitives (locks)
- Atomic operations
- Synchronization and messaging (channels)
- Multi-way concurrent control (select)

How to count right?

```
var myYoutubevideo struct {
    likes int32
}

func Viewer() {
    time.Sleep(time.Duration(rand.Intn(300)) * time.Millisecond)
    myYoutubevideo.likes = myYoutubevideo.likes + 1
}

func main() {
    for i := 0; i < 10000; i++ {
        go Viewer()
    }
    time.Sleep(3000 * time.Millisecond) // wait a long time
    fmt.Println(myYoutubevideo.likes, "likes")
}
```

What will be the result? inconsistent > race condition, threads access the same variable which is not always up to date, losing a few likes 17

Solution 1: Blocking Mutex

```
var myYoutubevideo struct {
    likes int32
    mu    sync.Mutex
}

func Viewer() {
    time.Sleep(time.Duration(rand.Intn(500)) * time.Millisecond)
    myYoutubevideo.mu.Lock()
    defer myYoutubevideo.mu.Unlock()
    myYoutubevideo.likes = myYoutubevideo.likes + 1
}

func main() {
    for i := 0; i < 10000; i++ {
        go Viewer()
    }
    for i := 0; i < 8; i++ {
        fmt.Println(myYoutubevideo.likes, "likes")
        time.Sleep(200 * time.Millisecond)
    }
}
```

Solution 2: Atomics

```
var myYoutubevideo struct {
    likes int32
}

func Viewer() {
    time.Sleep(time.Duration(rand.Intn(500)) * time.Millisecond)
    atomic.AddInt32(&myYoutubevideo.likes, 1)
}

func main() {
    for i := 0; i < 10000; i++ {
        go Viewer()
    }
    for i := 0; i < 8; i++ {
        fmt.Println(myYoutubevideo.likes, "likes")
        time.Sleep(200 * time.Millisecond)
    }
}
```

- Lock-free thread-safe programming on single variables
- Atomics either use a atomic CPU feature or perform a low level blocking operation via a mutex

Waitgroups (using global variablel)

```
func ParallelFor(n int, f func(int)) {
    wg.Add(n)
    for i := 0; i < n; i++ {
        go f(i)
    }
    wg.Wait()
}

func ProbablyPrime(value int) {
    if big.NewInt(int64(value)).ProbablyPrime(0) == true {
        fmt.Printf("%d is probably prime\n", value)
    }
    wg.Done()
}

func main() {
    fmt.Println("Start Program")
    ParallelFor(1000, ProbablyPrime)
    fmt.Println("Stop Program")
}
```

Waitgroups (using functional programming)

```
func ParallelFor(n int, f func(int)) {
    var wg sync.WaitGroup
    wg.Add(n)
    for i := 0; i < n; i++ {
        go func(i int) {
            defer wg.Done() // is always called before the goroutine exits
            f(i)
        }(i)
    }
    wg.Wait()
}

func ProbablyPrime(value int) {
    if big.NewInt(int64(value)).ProbablyPrime(0) == true {
        fmt.Printf("%d is probably prime\n", value)
    }
}

func main() {
    fmt.Println("Start Program")
    ParallelFor(1000, ProbablyPrime)
    fmt.Println("Stop Program")
}
```

Excercise

This is the previous code for atomic counting.

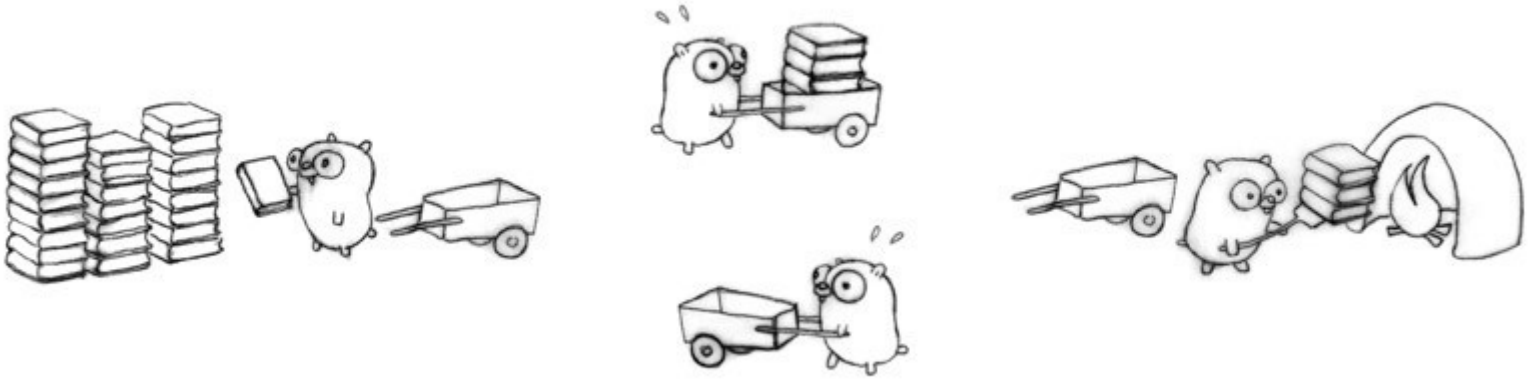
```
var myYoutubevideo struct {
    likes int32
}

func Viewer() {
    time.Sleep(time.Duration(rand.Intn(500)) * time.Millisecond)
    atomic.AddInt32(&myYoutubevideo.likes, 1)
}

func main() {
    for i := 0; i < 10000; i++ {
        go Viewer()
    }
    for i := 0; i < 8; i++ {
        fmt.Println(myYoutubevideo.likes, "likes")
        time.Sleep(200 * time.Millisecond)
    }
}
```

Use a waitgroup to wait for all threads to finish before printing the final result.

Go Channels



- Don't communicate by sharing memory; share memory by communicating (Rob Pike)

Channels

- Go routines can use channels for safe communication
- Construct a channel

```
c := make(chan int)     // buffer size = 0
c := make(chan int, 10) // buffer size = 10
```

- Send to channel

```
c <- 1
```

- Read from channel

```
x = <- c
```

- size = 0 (=default): Sender blocks until a reader requests a value from the channel
- size = n: Sender is not blocked until the buffer size is reached

Channels

Channels are typed values that allow goroutines to synchronize and exchange information.

```
func main() {
    timerChan := make(chan time.Time)

    go func() {
        time.Sleep(3 * time.Second)
        timerChan <- time.Now() // send time on timerChan
    }()

    fmt.Println("Started at", time.Now())
    // Do something else; when ready, receive.
    // Receive will block until timerChan delivers.
    // Value sent is other goroutine's completion time.
    completedAt := <-timerChan
    fmt.Println("Completed at", completedAt)
}
```

Lecturer 1: Channels

Let's use channels for communication:

```
func lecturer(c chan string) {
    for i := 0; i < 5; i++ {
        c <- fmt.Sprintf("%d Bla bla goroutines bla channels bla bla\n", i)
        time.Sleep(time.Duration(rand.Intn(1e3)) * time.Millisecond)
    }
}

func main() {
    c := make(chan string)
    go lecturer(c)
    for i := 0; i < 5; i++ {
        fmt.Printf(<-c)
    }
}
```

Lecturer 2: Channels

We can also return an (outgoing) channel instead of passing it as parameter:

```
func lecturer() <-chan string {
    c := make(chan string)
    go func() {
        for i := 0; i < 5; i++ {
            c <- fmt.Sprintf("%d Bla bla goroutines bla channels bla bla\n", i)
            time.Sleep(time.Duration(rand.Intn(1e3)) * time.Millisecond)
        }
    }()
    return c
}

func main() {
    c := lecturer()
    for i := 0; i < 5; i++ {
        fmt.Printf(<-c)
    }
}
```

Channels: Deadlocks

The following code might look good at first sight, but causes a deadlock:

```
package main

import "fmt"

func main() {
    ch := make(chan int)
    ch <- 1
    ch <- 2 // dead by now
    fmt.Println(<-ch)
    fmt.Println(<-ch)
}
```

Expected output?

28

Channels: Deadlocks

Same code, but with a channel with buffer size 2 (instead of 0):

```
package main

import "fmt"

func main() {
    ch := make(chan int, 2) // channel with buffer size 2
    ch <- 1
    ch <- 2 // dead by now
    fmt.Println(<-ch)
    fmt.Println(<-ch)
}
```

Channel and Errors

- Channel can be closed. Readers will return immediately. Successive writes will cause panic.

```
close(c)
```

- If a channel was closed, the reader gets "false" as return code (second return value)

```
value, ok := <-c
```

- Reading from a channel until closed

```
for {
    x, ok := <-c
    if !ok {
        break
    }
    // do something with x
}
// channel closed
```

Exercise 1: Generator

Write a generator for Fibonacci numbers, i.e. a function that returns a channel where the next Fibonacci number can be read.

```
func main() {
    fibChan := fib() // <- write func fib
    for n := 1; n <= 10; n++ {
        fmt.Printf("The %dth Fibonacci number is %d\n", n, <-fibChan)
    }
}
```

31

Lecturer 3: Anne & Bart

We're adding another (slower) lecturer to make it more interesting:

```
func lecturer(name string, speed int) <-chan string {
    c := make(chan string)
    go func() {
        for i := 0; i < 5; i++ {
            c <- fmt.Sprintf("%s: %d Bla bla goroutines bla channels bla bla\n", name, i)
            time.Sleep(time.Duration(rand.Intn(1e3*speed)) * time.Millisecond)
        }
    }()
    return c
}

func main() {
    a := lecturer("Anne", 1)
    b := lecturer("Bart", 2)
    for i := 0; i < 5; i++ {
        fmt.Printf(<-a)
        fmt.Printf(<-b)
    }
}
```

Lecturer 4: Fan In

```
// func lecturer(name string, speed int) <-chan string { ... }

func fanIn(c1, c2 <-chan string) <-chan string {
    c := make(chan string)
    go func() { for { c <- <-c1 } }()
    go func() { for { c <- <-c2 } }()
    return c
}

func main() {
    a := lecturer("Anne", 1)
    b := lecturer("Bart", 2)
    c := fanIn(a, b)
    for i := 0; i < 10; i++ {
        fmt.Printf(<-c)
    }
}
```

Lecturer 5: Select

```
func lecturer(name string, speed int) <-chan string {
    c := make(chan string)
    go func() {
        for i := 0; i < 5; i++ {
            c <- fmt.Sprintf("%s: %d Bla bla goroutines bla channels bla bla\n", name, i)
            time.Sleep(time.Duration(rand.Intn(1e3*speed)) * time.Millisecond)
        }
    }()
    return c
}

func main() {
    a := lecturer("Anne", 1)
    b := lecturer("Bart", 2)
    for i := 0; i < 10; i++ {
        select {
        case msgFromAnne := <-a: fmt.Printf(msgFromAnne)
        case msgFromBart := <-b: fmt.Printf(msgFromBart)
        }
    }
}
```

34

Select

The `select` statement is like a `switch`, but the decision is based on ability to communicate rather than equal values.

```
select {
    case v := <-ch1:
        fmt.Println("channel 1 sends", v)
    case v := <-ch2:
        fmt.Println("channel 2 sends", v)
    default: // optional
        fmt.Println("neither channel was ready")
}
```

Without default case, the select blocks until a message is received on one of the channels.

Exercise 2: Timeout

Write a function `setTimeout()` that times out an operation after a given amount of time. Hint: Have a look at the built-in `time.After()` function and make use of the `select` statement.

```
func main() {
    res, err := setTimeout(
        func() int {
            time.Sleep(2000 * time.Millisecond)
            return 1
        }, 1*time.Second)

    if err != nil {
        fmt.Println(err.Error())
    } else {
        fmt.Printf("operation returned %d", res)
    }
}
```

Go really supports concurrency

Really.

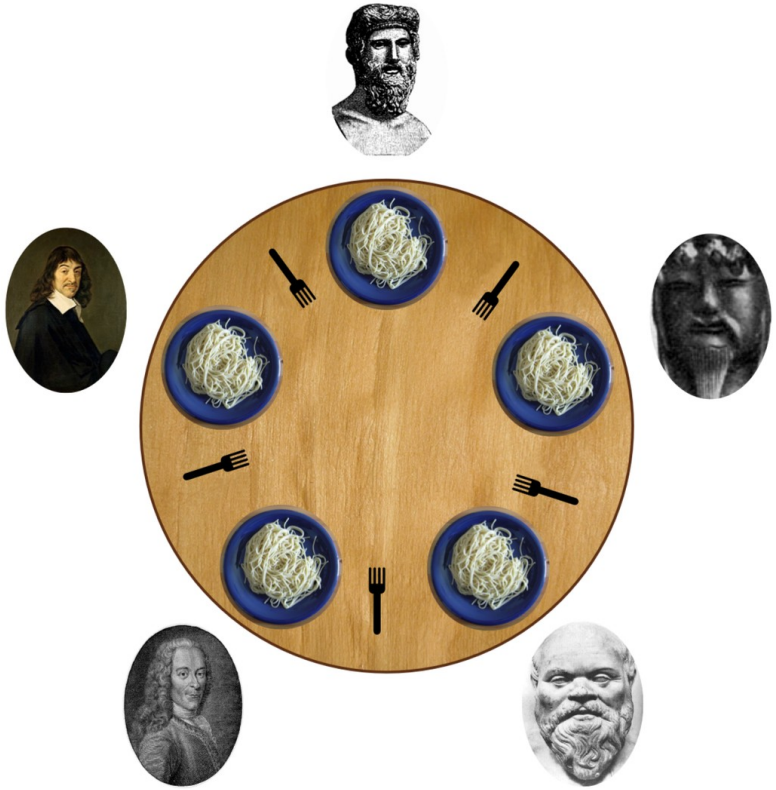
- It's routine to create thousands of goroutines in one program. (not unusual to debug a program after it had created even millions goroutines)
- With Go you can solve sync problems with channels
- Channels use Message Passing instead of locks

More information about Go and concurrency:

youtu.be/f6kdp27TYZs?t=1 (https://youtu.be/f6kdp27TYZs?t=1) 37

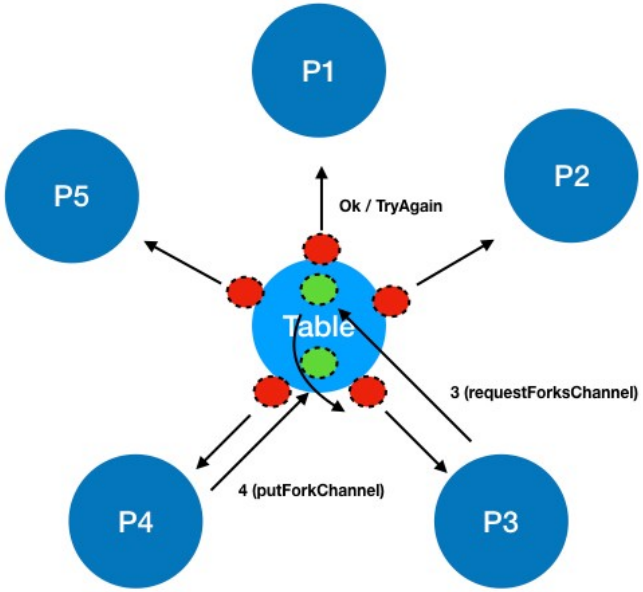
Dining Philosophers

Exercise 3: Dining Philosophers



5 philosophers think either or eat. To eat the spaghetti each need 2 forks.
A hungry philosopher sits down, takes the right fork, then the left, eats, then puts the left fork on the table, then the right. Everyone uses only the one to the left or to the right of his place.

Dining Philosophers with Channels



- Philosopher asks for the forks over requestForksChannel (green)
- Waits for the reserved channel if the operation was successful (red).
- Repeats if not successful
- Philosopher puts the forks back via the putForkChannel

40

Dining Philosophers - Hints

- The table itself should be a Go Routine and should handle all forks in a single thread. This makes synchronization easy.
- The philosopher itself should be kept simple. It uses mainly the API of the table
- Never grab one fork and wait for the other. This is a deadlock situation.
- If you can't get the second fork, you should immediately release the first one.
- The philosopher loop looks like this:

```
// Main loop
func (p *Philosopher) run() {
    for {
        p.takeForks()
        p.eat()
        p.putForks()
        p.think()
    }
}
```

play.golang.org/p/rXCotNNY24 (https://play.golang.org/p/rXCotNNY24)

Thank you

Tags: go, programming, concurrent, go routines, channels (#ZgotmplZ)

Sebastian Macke
Rosenheim Technical University
Sebastian.Macke@th-rosenheim.de (mailto:Sebastian.Macke@th-rosenheim.de)
https://www.qaware.de (https://www.qaware.de)